ОСНОВЫ МАШИННОГО ОБУЧЕНИЯ · ДОПОЛНИТЕЛЬНЫЕ БЛОКИ

Текст как данные

Как язык попадает в машинное обучение и что с ним там происходит

Магистратура «Цифровая лингвистика и локализация». Все числа на слайдах — из лабораторных работ курса, на том же корпусе из 160 сегментов локализации.

Пять блоков и где их потрогать руками

Основной курс объясняет машинное обучение на числах. Эти блоки — о том, как в эту схему попадает текст. Каждый привязан к лабораторной и к странице, где то же самое можно покрутить руками.

блок	о чём	работа	playground
1	текст как данные: токены, словоформы, признаки	Л.р. № 1	векторизация
2	математический минимум и обучение модели по шагам	Л.р. № 1 и № 2	градиентный спуск
3	языковые модели	Л.р. № 4	языковая модель
4	метрики для текста	Л.р. № 3	метрики перевода
5	данные и разметка	Л.р. № 1 и № 3	данные и разметка

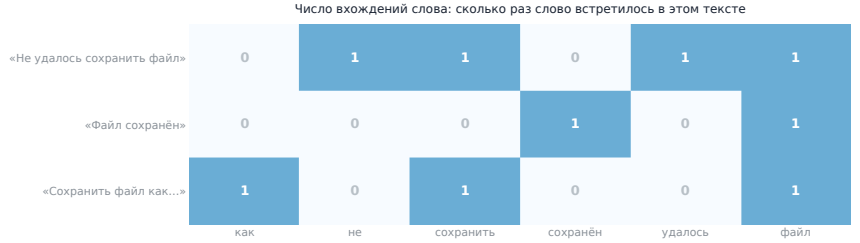
Все playground'ы — на konkin-nikita.ru/gd_learn: они считают те же числа на том же корпусе, что и ноутбуки.

БЛОК 1

Текст как данные

Модель видит числа. Всё, что вы знаете о языке, попадает в неё ровно одним способом — через то, как вы превратили текст в числа.

Модель не читает текст



«сохранить» и «сохранён» — два разных столбца: машина не знает, что это формы одного слова

Любой алгоритм принимает таблицу: строка — объект, столбец — признак. Между текстом и моделью всегда стоит шаг векторизации, и проектирует его человек: выбор единицы, нормализация, отбор признаков — всё это решения о языке.

Л.р. № 1, раздел 8. Один и тот же наивный Байес на одном и том же корпусе даёт от **0,381** до **0,631**. Меняется только способ превратить текст в признаки.

Шаг первый: токенизация

Разбить текст на единицы. Кажется тривиальным — до первого реального файла.

Пунктуация: «серверу.» и «серверу» — одна единица или две?

Дефисы: «из-за», «кто-нибудь», «интернет-магазин».

Плейсхолдеры: `{name}`, `%s`, `<b>` — их нельзя ни резать, ни переводить.

Числа и единицы: «25 МБ», «02:00», «TLS 1.3».

Буква ё: «все» и «всё» — разные слова.

Л.р. № 1, раздел 4. Словарь корпуса начинается с `00`, `02`, `25`, `429`, `api`, `csv`, `https`, `minutes`, `name`. Токенизатор по умолчанию разрезал время и превратил плейсхолдеры `{minutes}` и `{name}` в английские слова.

Готовые токенизаторы: `razdel`, `nltk`, `spaCy`. Регулярное выражение годится для учебной задачи и ломается на реальной.

Шаг второй: нормализация словоформ

Стемминг

Усечение до основы по правилам. Быстро, грубо, без словаря; основа не обязана быть словом.

`сохранены`, `сохранить` → `сохран`
`изменения` → `изменен`

Лемматизация

Приведение к словарной форме. Нужны словарь и снятие омонимии: «стали» — это «стать» или «сталь»?

Для русского: `pymorphy3`, `spaCy`, `Natasha`.

— вывод раздела 8 Л.р. № 1

Для английского разница невелика. Для русского принципиальна: одна лексема даёт десятки словоформ, и без нормализации модель считает их не связанными между собой признаками.

Л.р. № 1: стемминг поднимает точность наивного Байеса с 0,444 до 0,538 — но на разных частях кросс-валидации результат расходится до 0,135. Одна таблица этого не доказывает; доказывают сегменты, которые он исправил.

Шаг третий: мешок слов и TF-IDF

Мешок слов

Словарь корпуса и число вхождений каждого слова. Порядок теряется полностью: «не удалось подключиться» и «удалось не подключиться» — один и тот же вектор.

TF-IDF

Вес слова — произведение двух множителей: как часто оно в этом тексте (TF) и насколько редко по корпусу (IDF). То, что есть везде, весит почти ноль.

TF-IDF — не «улучшенный мешок слов», а другой ответ на вопрос, что считать важным. В разделе 6 Л.р. № 1 на одном разбиении он даже проигрывает: 0,400 против 0,425. Это один сегмент из сорока — то есть ничего, и ровно поэтому дальше в работе одна пятая корпуса по очереди становится контрольной.

TF-IDF по формуле

$$w(t,d) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t)$$

$$\mathrm{idf}(t) = \ln \frac{1+N}{1+\mathrm{df}(t)} + 1$$

tf — сколько раз слово t стоит в сегменте d;
df — в скольких сегментах оно есть;
$N = 160$ — сегментов в корпусе.

Затем вектор делят на его длину: так длинный и короткий сегменты становятся сравнимы.

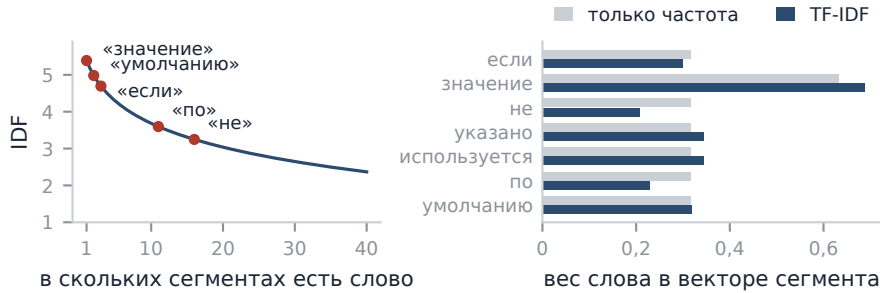
s044: «Если значение не указано, используется значение по умолчанию.»

слово	tf	df	idf	tf·idf	вес
значение	2	1	5,39	10,78	**0,689**
указано	1	1	5,39	5,39	0,345
используется	1	1	5,39	5,39	0,345
умолчанию	1	2	4,98	4,98	0,319
если	1	3	4,70	4,70	0,300
по	1	11	3,60	3,60	0,230
не	1	16	3,25	3,25	**0,208**

вес = tf·idf / 15,64, где
$15{,}64 = \sqrt{10{,}78^2 + 5{,}39^2 + \ldots + 3{,}25^2}$ — длина вектора

TF-IDF измеряет редкость, а не важность: «не» весит втрое меньше «значения», хотя для перевода отрицание важнее всего.

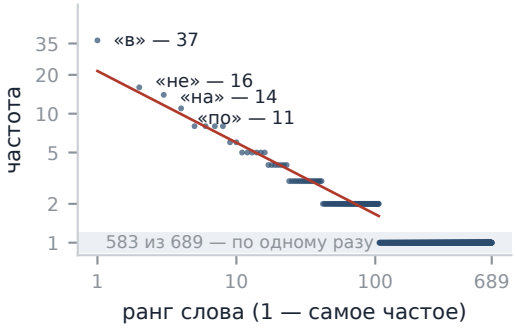
TF-IDF на графике: редкое весит больше



Слева — IDF при $N = 160$: слово из одного сегмента весит 5,39, из 16 — уже 3,25. Справа — s044: серым — веса по частоте, синим — с IDF. Редкие слова поднялись, частые «не» и «по» опустились.

Признаков-слов в Л.р. № 1 — 679, а словоформ в корпусе 689: `TfidfVectorizer` по умолчанию отбрасывает словоформы из одного знака. Для русского это предлоги и союзы «в», «с», «к», «о», «у», «и», «а» — и три цифры.

Почему векторы почти пустые: закон Ципфа



Частота слова примерно обратно пропорциональна его месту r в частотном списке:

$$f(r) \approx \frac{C}{r^a}$$

В логарифмическом масштабе — прямая с наклоном $-a$. В больших корпусах $a \approx 1$, на наших 947 словах — 0,55.

Главное — хвост: **583** словоформы из 689 (85 %) встретились по разу.

Для модели это значит: почти каждое слово нового сегмента она видит впервые. Отсюда пустые векторы на следующем слайде и выигрыш n-грамм: часть слова повторяется, даже когда само слово — нет.

Шаг четвёртый: а нужно ли слово вообще

Символьные n-граммы: признак — подстрока из нескольких букв. Понятия «слово» в модели нет.

«сохранить» → сох, охр, хра, ран, ани, нит, ить

«сохранение» даёт те же сох, охр, хра, ран — словоформы похожи без словаря и без правил.

Тот же наивный Байес:

слова	0,444
стемминг	0,538
символьные 2–4	0,612
символьные 3–5	**0,631**

Цена: 5418–7821 признаков вместо 679, и прочесть их человек не может.

Почему слова проигрывают на коротких строках. У 44 из 160 сегментов, когда они попадают в контрольную часть, нет ни одного слова, встречавшегося при обучении: вектор пустой. Пустых векторов из n-грамм нет ни одного.

→ **векторизация текста** — единица, стемминг, n-граммы и TF-IDF переключаются мышью.

БЛОК 2

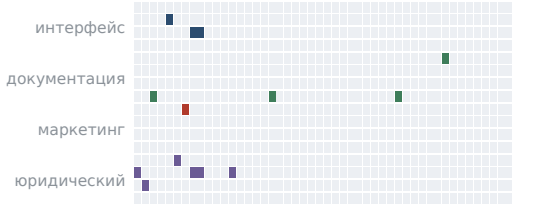
Математический минимум

Пять понятий, на которых держится всё остальное: вектор, косинус, модель, потери, градиент. Каждая формула — с расшифровкой и с числами нашего корпуса.

Вектор — это просто список чисел

Вектор длины n — упорядоченный список из n чисел: $\mathbf{x} = (x_1, \ldots, x_n)$. И всё.

После векторизации сегмент — точка в пространстве, где измерений столько, сколько признаков. Представить 5418 измерений нельзя, и не нужно: всё, что мы делаем с векторами, определено для любого их числа.

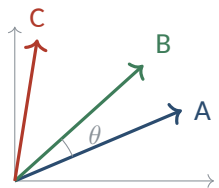


Корпус глазами модели, фрагмент: 16 сегментов × 48 случайных n-грамм. Закрашено 14 клеток из 768.

Похожие тексты — близкие точки.

Корпус Л.р. № 1 после символьных n-грамм 2–4 — **160** точек в **5418**-мерном пространстве. Ненулевых координат **2,1 %**: в среднем 115 на сегмент, у коротких строк интерфейса — 63.

Скалярное произведение и косинус



A и B смотрят почти в одну сторону, C — в другую.

Скалярное произведение — перемножить числа попарно и сложить. Велико, если у обоих векторов большие числа на одних и тех же местах:

$$\mathbf{a} \cdot \mathbf{b} = a_1 b_1 + a_2 b_2 + \ldots + a_n b_n$$

Косинус — то же, делённое на длины векторов:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}, \qquad \|\mathbf{a}\| = \sqrt{a_1^2 + \ldots + a_n^2}$$

Для TF-IDF он от 0 до 1: единица — одинаковый состав, ноль — ни одного общего признака. Длина текста не влияет — в отличие от Левенштейна.

Л.р. № 2, разделы 3–4: на 13 запросах к памяти переводов косинус и Левенштейн расходятся в 4 случаях, и в двух из них можно обосновать оба ответа.

→ **память переводов** — четыре меры похожести на одном запросе.

Косинус вручную

Признаки — слова, значение — сколько раз слово встретилось

A = «Не удалось подключиться к серверу», **B** = «…к базе данных»

	базе	данных	к	не	подключиться	серверу	удалось
A	0	0	1	1	1	1	1
B	1	1	1	1	1	0	1

$$\mathbf{A}\cdot\mathbf{B} = 0\cdot1+0\cdot1+1\cdot1+1\cdot1+1\cdot1+1\cdot0+1\cdot1 = 4$$

$$\|\mathbf{A}\| = \sqrt{5}, \quad \|\mathbf{B}\| = \sqrt{6}, \qquad \cos(\mathbf{A},\mathbf{B}) = \frac{4}{\sqrt{5}\cdot\sqrt{6}} = \frac{4}{\sqrt{30}} \approx 0{,}730$$

«Настройки поиска» и «Поиск по настройкам»: общих слов нет, и косинус по словам — ровно 0. По символьным n-граммам 2–4 общих 34 из 45 и 48, косинус — **0,731**. Один смысл без общих словоформ — то самое расхождение мер, которое разбирается в Л.р. № 2.

Что значит «обучить модель»

У модели есть **веса** — числа, которые она может менять.

Есть **функция потерь** — число, показывающее, насколько предсказания расходятся с правильными ответами.

Обучение — подобрать веса так, чтобы потери стали меньше. Никакого понимания в этом нет: модель не «выучила язык», она подобрала числа.

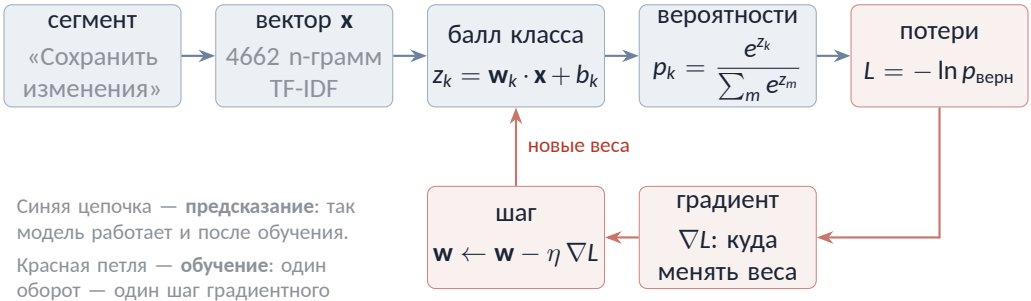
Отсюда главное правило: модель проверяют на данных, которых она не видела.

В Л.р. № 1 (раздел 12) лучшая модель на своих обучающих сегментах даёт **1,000**, на отложенных — **0,744**. Первое число измеряет память, второе — способность обобщать.

Модель воспроизводит то, что было в обучающих данных, — вместе с их ошибками и перекосами. К этому мы вернёмся в блоке 5.

Обучение классификатора: весь цикл на одной схеме

Дальше каждый блок схемы — отдельный слайд, на корпусе Л.р. № 1



Синяя цепочка — **предсказание**: так модель работает и после обучения.

Красная петля — **обучение**: один оборот — один шаг градиентного спуска по всем 120 обучающим сегментам сразу.

Петля крутится, пока потери не перестанут падать; на нашем корпусе — 3000 оборотов. Весов, которые она подбирает, $4 \times 4662 = 18\,648$, и до первого оборота все они — нули.

Модель: балл класса — взвешенная сумма n-грамм

$$z_k = \sum_{j=1}^{4662} w_{kj}\, x_j + b_k$$

x_j — TF-IDF n-граммы j в сегменте; у большинства — ноль.
w_{kj} — насколько n-грамма j говорит «за» класс k: плюс — за, минус — против.
b_k — сдвиг (свободный член): балл класса, когда признаков нет.

Это скалярное произведение: вектор сегмента на вектор весов класса.

Сколько тут чисел

У каждого из 4 классов свой вес для каждой из 4662 n-грамм:
$4 \times 4662 = 18\,648$ весов и 4 сдвига.
До обучения все они — нули.

Ответ модели — класс с наибольшим баллом. Учить её удобнее через вероятности — следующий слайд.

Пример. У отложенного сегмента «Компания оставляет за собой право…» 192 n-граммы. Их вклады в балл юридического класса в сумме дают 5,97, сдвиг — $-1{,}62$, итого $z = 4{,}35$.

Softmax: от баллов к вероятностям

$$p_k = \frac{e^{z_k}}{e^{z_1} + e^{z_2} + e^{z_3} + e^{z_4}}$$

Экспонента делает каждый балл положительным, деление на сумму — сумму вероятностей равной 1. Порядок не меняется: больший балл — большая вероятность.

Отложенный сегмент после обучения: «Компания оставляет за собой право в любое время вносить изменения в настоящие Условия.»

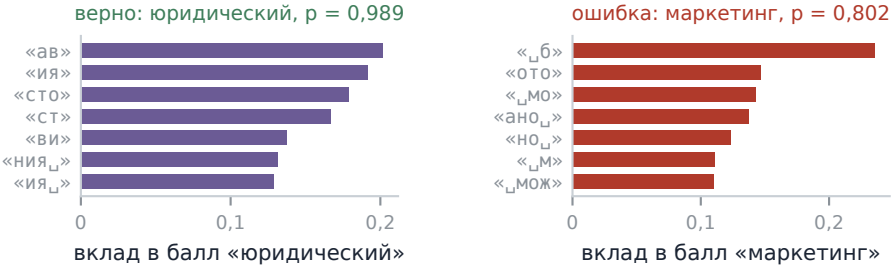
класс	z_k	e^{z_k}	p_k
интерфейс	−1,62	0,20	0,003
документация	−0,69	0,50	0,006
маркетинг	−2,04	0,13	0,002
юридический	4,35	77,45	**0,989**
сумма		78,28	1

$p_{\text{юр}} = 77{,}45 / 78{,}28 = 0{,}989$

Зачем именно экспонента. Она усиливает разрывы: баллы юридического класса и документации различаются на 5,04, а вероятности — в $e^{5{,}04} \approx 155$ раз.

Почему модель так решила

Вклад n-граммы — её вес, умноженный на её TF-IDF: $w_{kj}\, x_j$; семь самых больших



«Компания оставляет за собой право…»: больше всего баллов дали части слов «право», «изменения», «настоящие Условия».

«Согласие может быть отозвано…» — юридический, а ответ «маркетинг»: «␣б» из «быть», как в «бесплатно»; «␣мож» из «может», как в «можно».

Модель складывает части слов, а не понимает их, — зато её ошибку можно объяснить.

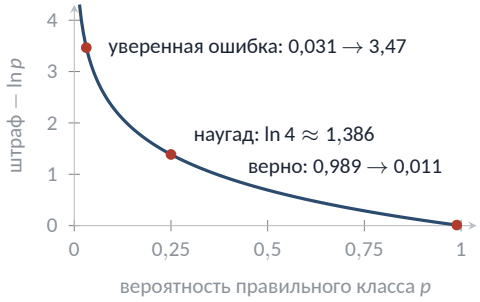
Функция потерь: перекрёстная энтропия

Берём вероятность, которую модель дала **правильному** классу; штраф за сегмент — её логарифм со знаком минус. Потери — средний штраф:

$$L = -\frac{1}{N}\sum_{i=1}^{N} \ln p_{y_i}(\mathbf{x}_i)$$

$N = 120$ обучающих сегментов, y_i — правильный класс i-го.

При $p = 1$ штраф ноль; чем меньше p, тем он больше — и без предела.



До обучения все веса — нули, каждому классу $p = \frac{1}{4}$, поэтому обучение стартует с $L = \ln 4 \approx 1{,}386$. Ошибка на «Согласие может быть отозвано…» стоит 3,47 — в 300 с лишним раз дороже верного ответа с $p = 0{,}989$.

Градиент: как именно подбирают веса

Перебрать все комбинации весов невозможно. **Градиент** отвечает на вопрос: если чуть-чуть изменить каждый вес, в какую сторону потери убывают быстрее всего? Дальше — шаг в эту сторону, пересчёт, снова шаг. Это градиентный спуск: спуск по склону в тумане, на ощупь.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla L(\mathbf{w})$$

∇L — для каждого из 18 648 весов число «насколько вырастут потери, если этот вес чуть увеличить»; η (эта) — размер шага; минус — потому что идём вниз.

Размер шага

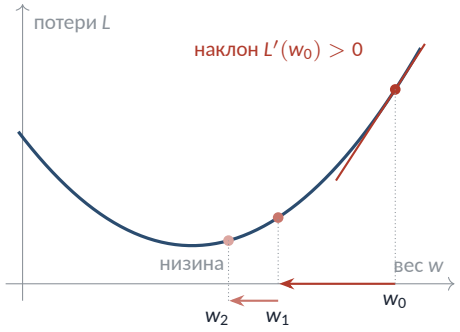
Слишком велик — модель перескакивает низину, потери скачут вверх. Слишком мал — обучение не заканчивается. SGD, Adam, RMSprop — разные правила того, как делать шаг.

Что из этого следует

Обучение — пошаговый подбор: результат зависит от начальной точки и от случайности. Выводить формулу градиента не нужно; читать её полезно — это следующие слайды.

→ **градиентный спуск** — шаги видны на поверхности потерь, и сразу заметно, когда спуск расходится.

Градиент на одном весе



Схема, а не данные: потери как функция одного веса.

Меняется только один вес w. Производная $L'(w)$ — наклон кривой потерь в текущей точке:

$$w_{\text{новый}} = w - \eta\, L'(w)$$

Наклон больше нуля — потери растут вправо, идём влево; меньше нуля — вправо.

Шаг пропорционален наклону: на склоне длинный, у дна короткий — спуск сам замедляется у минимума.

Градиент ∇L — 18 648 таких наклонов сразу, по одному на вес.

Градиент нашей модели: ошибка × признак

Для логистической регрессии наклон по каждому весу считается одной формулой:

$$\frac{\partial L}{\partial w_{kj}} = \frac{1}{N}\sum_{i=1}^{N} \underbrace{\big(p_{ik} - [\,y_i = k\,]\big)}_{\text{ошибка в вероятности}} \cdot \underbrace{x_{ij}}_{\text{признак}}$$

$[y_i = k]$ — единица, если k — правильный класс i-го сегмента, иначе ноль. У правильного класса ошибка отрицательна (вероятность занижена), у остальных — положительна (завышена).

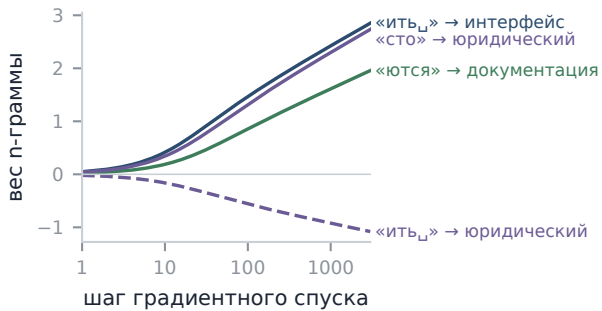
Что из формулы следует

Вес n-граммы меняют только сегменты, где она есть: при $x_{ij} = 0$ слагаемое — ноль. Вес растёт у класса, которому модель недодаёт вероятность, и падает у остальных.

Первый шаг на нашем корпусе

Все $p = \frac{1}{4}$. У каждой строки интерфейса с «ить␣» («Удалить…», «Отменить…») ошибка $\frac{1}{4} - 1 = -\frac{3}{4}$ у интерфейса и $+\frac{1}{4}$ у остальных классов. После шага вес «ить␣» для интерфейса — $+0{,}048$, для юридического — $-0{,}019$.

Как растут веса: модель находит инфинитив



Четыре веса из 18 648 за 3000 шагов.

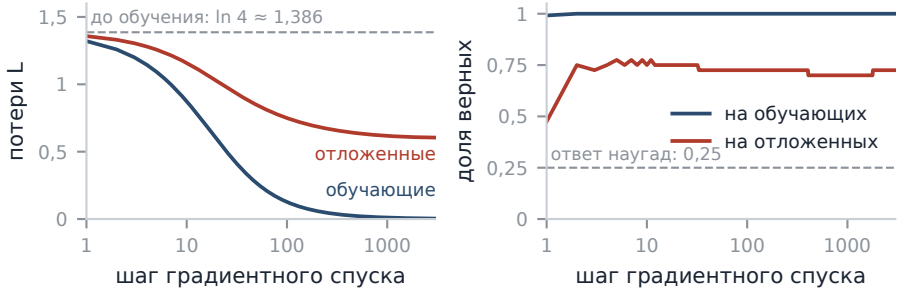
«ить␣» — конец инфинитива («удалить»). Для интерфейса вес вырос до 2,85, для юридического упал до −1,08.

«ются» («удаляются») — документация, 1,96; «сто» («стороны») — юридический, 2,73.

Про инфинитив модели никто не говорил. Она нашла его по градиенту: вес растёт, пока сегменты с этой n-граммой недополучают вероятность своего класса.

Обучение на корпусе: потери и точность по шагам

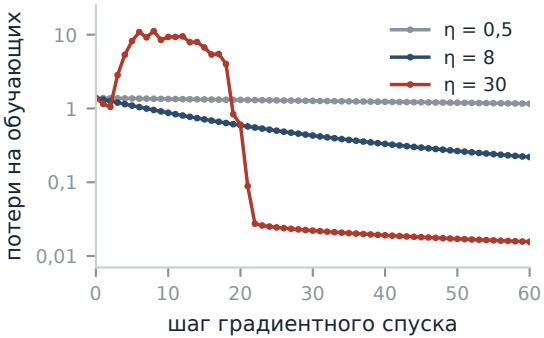
120 обучающих и 40 отложенных сегментов, $\eta = 8$, ось шагов логарифмическая



Старт — $\ln 4$. Ко 2-му шагу на обучающих верно всё, на отложенных — 0,750 (лучшее — 0,775, к концу — 0,725). Потери на обучающих уходят к нулю, на отложенных застревают на 0,605: разрыв — это то, что модель запомнила наизусть, но не обобщила.

sklearn по умолчанию добавляет к L штраф за большие веса $\frac{\lambda}{2}\sum w^2$ (регуляризацию). С ним этот спуск приходит к весам `LogisticRegression` и даёт 0,750, как во «Введении».

Размер шага на нашем корпусе



η = 0,5: за 60 шагов потери 1,39 → 1,16 — медленно.

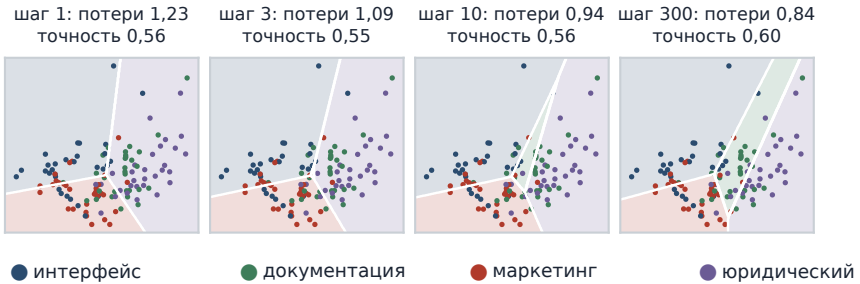
η = 8: потери ровно убывают, 0,22 к 60-му шагу.

η = 30: перескочил низину — к 8-му шагу потери 11,15, в 8 раз выше старта; выбрался к 20-му. То, что потом красная линия ниже синей, — не победа: потери на обучающих измеряют память.

Здесь повезло, и это не правило. Со штрафом за веса, как в sklearn по умолчанию, спуск с шагом 30 так и не выправляется: через 3000 шагов потери 10,8, точность на отложенных 0,25 — как при ответе наугад.

Обучение, которое можно нарисовать

Тот же спуск на двух признаках: n-граммы сжаты до двух координат LSA, как в Л.р. № 2

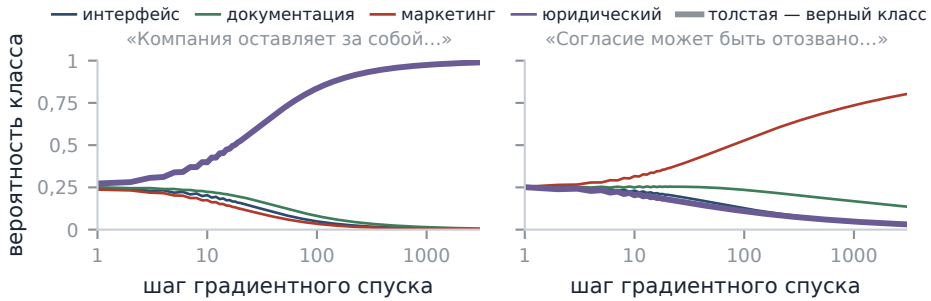


Точки — обучающие сегменты; фон — класс, который модель назовёт в этой точке; белые линии — границы. Область документации появляется только к 10-му шагу и растёт клином между интерфейсом и юридическими текстами.

Границы прямые, потому что модель линейная. На двух координатах точность 0,600 на обучающих и 0,625 на отложенных — против 0,725 на всех n-граммах: при сжатии часть различий потеряна.

Как растёт уверенность модели

Вероятности четырёх классов для двух отложенных сегментов

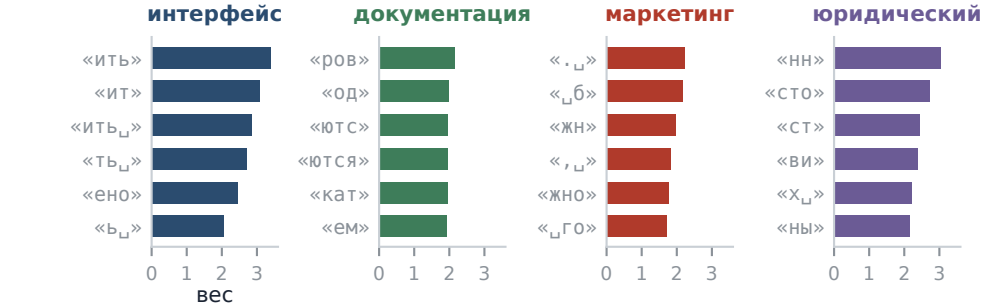


Слева: у юридического к 10-му шагу 0,398, к 3000-му — 0,989. Чем дольше обучение, тем увереннее верный ответ.

Справа: тот же механизм так же старательно растит ошибку — у маркетинга 0,802, у верного юридического 0,031.

Уверенность — не правота: 0,8 у ошибки значит лишь, что n-граммы похожи на маркетинговые. Из 40 отложенных модель ошиблась на 11.

Что модель выучила: самые весомые n-граммы



Шесть n-грамм с наибольшим весом для каждого класса после 3000 шагов. Знак ␣ — граница слова: «ить␣» — «ить» в конце слова.

Модель видела только сочетания букв и метки классов. Всё, что на этой картинке похоже на грамматику, она нашла сама — а вместе с грамматикой и шум. Отделить одно от другого может лингвист; как — на следующем слайде.

Как прочесть эти веса лингвисту

класс	n-граммы	слова обучающей части	что это
интерфейс	«ить», «ить␣»	удалить, отменить, добавить	инфинитив: команды
	«ено»	восстановлено, найдено	краткое причастие
документация	«ются», «ются»	удаляются, создаются	возвратные формы
	«ров», «кат»	параметров, сертификат	части терминов
маркетинг	«␣б», «жно»	бесплатно, можно, нужно	выгода, модальность
	«.␣», «,␣»	—	короткие фразы
юридический	«сто», «нн»	стороны, настоящий, данные	лексика договора
	«х␣»	данных, условиях	мн. ч., род. и предл. п.

Модель нашла морфологию, которую назвал бы и лингвист. Но «ров» и «кат» — не признак регистра, а совпадение на 120 примерах. Отличить одно от другого может только человек: читать веса — работа лингвиста.

БЛОК 3

Языковые модели

От n-грамм до трансформера: что изменилось и почему это важно для вашей работы.

Задача языковой модели не менялась с 1950-х

Предсказать следующую единицу по предыдущим. Единицей может быть символ, слово или часть слова; на выходе — распределение вероятностей по всему словарю.

Из этой задачи вырастает всё остальное: генерация, подсказки при наборе, перевод, ответы на вопросы.

В постановке нет ничего про смысл. Модель учится делать продолжение вероятным — похожим на тексты, на которых обучалась, — а не истинным, уместным или полезным. Всё, что выглядит как понимание, — следствие статистики на очень больших объёмах текста.

Как из распределения выбирают символ — температура, top-k, top-p — разобрано в лекции к Л.р. № 4.

→ **языковая модель и температура** — распределение следующего символа видно целиком.

Три поколения

N-граммы

до ∼2010

Таблица частот: какие единицы за какими шли. Просто и быстро; контекст — несколько единиц, дальше данных не хватает.

Рекуррентные сети

2010–2017

Сеть читает слово за словом, неся память о прочитанном. Начало длинного текста быстро забывается; обучать можно только последовательно.

Трансформеры

с 2017

Внимание: каждая позиция напрямую видит все остальные. Дальний контекст не теряется, обучение параллельно — отсюда огромные модели.

Предел первого поколения видно на нашем корпусе. Символьная n-граммная модель при $n = 1$ пишет оригинальную бессмыслицу, при $n = 8$ — гладкий текст, на 100 % списанный из корпуса. Перплексия на отложенных текстах минимальна при $n = 3$. «Складно» и «выучено наизусть» растут вместе.

N-граммная модель в формулах

Вероятность текста раскладывается в цепочку: каждый символ — при условии всех предыдущих.

$$P(c_1 c_2 \dots c_T) = P(c_1) \cdot P(c_2 \mid c_1) \cdot P(c_3 \mid c_1 c_2) \cdot \dots \cdot P(c_T \mid c_1 \dots c_{T-1})$$

Модель порядка n помнит только n предыдущих символов, а вероятность оценивает частотами в обучающем тексте:

$$P(c_t \mid c_{t-n} \dots c_{t-1}) \;\approx\; \frac{\#(c_{t-n} \dots c_{t-1}\, c_t)}{\#(c_{t-n} \dots c_{t-1})}$$

$\#(\dots)$ — сколько раз цепочка встретилась. Обучение здесь — подсчёт, градиента нет.

Пример, $n = 3$**.** В обучающем тексте после «сох» всегда шло «р»: «сохранены», «сохраняют», «сохранения». Модель ставит «р» с вероятностью **0,921**, ничего не зная о слове.

Сглаживание. Цепочка, которой не было, получила бы вероятность ноль. Поэтому оценку смешивают с более короткими контекстами; на интерактивной странице длинный контекст берётся с весом 0,7.

Перплексия: формула и кривая на нашем корпусе

$$\mathrm{PP} = 2^{-\frac{1}{T}\sum_{t=1}^{T} \log_2 P(c_t \mid \text{контекст})}$$

В показателе степени — средний штраф $-\log P$ за правильный символ, то есть перекрёстная энтропия из блока 2. $\mathrm{PP} = 10$: модель сомневается, как при выборе из 10 равновероятных символов.



На обучающем перплексия падает всё время. На отложенном — 32 сегментах из 160, которых модель не видела, — она минимальна при $n = 3$ и дальше растёт, а сгенерированный текст всё больше списан из корпуса. Та же картина, что у классификатора в блоке 2: успех на обучающих измеряет память.

→ **языковая модель** — порядок и температура меняются ползунками.

Что дало внимание

В рекуррентной сети, чтобы связать слово с тем, что было двадцать слов назад, информация проходит двадцать шагов и на каждом частично теряется. Внимание убирает цепочку: каждая позиция сразу получает доступ ко всем остальным и сама решает, на какие смотреть.

Что стало возможно

Согласование на большом расстоянии перестаёт быть проблемой; разрешение анафоры становится в принципе достижимым.

Чем за это платят

Вычисления растут квадратично от длины текста: вдвое более длинный документ стоит вчетверо дороже.

Для лингвиста главное вот что: модель научилась учитывать структуру предложения, не имея никакого её описания. Синтаксис в неё не закладывали.

Внимание в формуле: тот же softmax

У каждого слова i три вектора: запрос $\mathbf{q}_i$ («что я ищу»), ключ $\mathbf{k}_i$ («что я могу предложить») и значение $\mathbf{v}_i$ («что я передам»). С каким весом слово i смотрит на слово j:

$$a_{ij} = \mathrm{softmax}_j\left(\frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d}}\right)$$

Новое представление слова i — смесь значений с этими весами:

$$\mathbf{h}_i = \sum_j a_{ij}\,\mathbf{v}_j$$

Все детали уже встречались. Скалярное произведение измеряет, насколько ключ подходит к запросу, — как в косинусе. Softmax превращает баллы в веса, в сумме дающие 1, — как в классификаторе. $\sqrt{d}$, корень из длины векторов, не даёт баллам слишком вырасти.

Сами $\mathbf{q}$, $\mathbf{k}$, $\mathbf{v}$ получаются умножением вектора слова на матрицы весов. Их и подбирает градиентный спуск.

Откуда квадрат. Для текста из n слов нужно $n \times n$ скалярных произведений: каждое слово с каждым. Текст вдвое длиннее — работы вчетверо больше.

Почему модели берут готовыми

Предобучение

Месяцы вычислений на миллиардах слов. Вам недоступно — и не нужно.

Дообучение

Часы на вашей задаче и вашей тысяче примеров.

Вывод для локализации: свою модель перевода вы обучать не будете. Вы будете брать готовую, оценивать её на своих данных и решать, годится ли она.

Это не упрощение курса, а описание реальной работы. Поэтому центральная лабораторная — не про обучение, а про оценку.

Л.р. № 3 работает ровно по этой схеме. Машинный перевод корпуса сделан готовой моделью `Helsinki-NLP/opus-mt-en-ru` (раздел 10 работы); студент ничего не обучает, он измеряет.

БЛОК 4

Метрики для текста

Как измеряют качество перевода, что при этом не измеряется и где метрика вводит в заблуждение.

Почему «совпал — не совпал» здесь не работает

В классификации ответ либо совпал с правильным, либо нет. Для перевода так нельзя: «Сохранить изменения» и «Сохранение изменений» допустимы оба, а точное совпадение строк дало бы ноль за верный перевод.

Значит, нужна метрика *степени* совпадения с эталоном.

Главное ограничение всего подхода: эталон один, а верных переводов много. Всё, что метрика может сказать, — насколько оцениваемый перевод — его называют *гипотезой* — похож на один конкретный человеческий вариант.

BLEU: что он на самом деле считает

Для $n = 1, 2, 3, 4$ берётся доля n-грамм гипотезы, нашедшихся в эталоне. Четыре доли усредняются **геометрически**, результат умножается на штраф за слишком короткий ответ. Смысла, грамматичности, синонимов и словоформ в этом определении нет.

эталон	гипотеза	BLEU
Не удалось подключиться к серверу	та же строка	1,000
Не удалось подключиться к серверу	**Удалось** подключиться к серверу	**0,779**
Сохранить изменения	Сохранение изменений	0,408

Противоположный смысл — 0,779, верный перевод другими формами — 0,408. **BLEU считает форму, а не смысл.** Л.р. № 3, раздел 3.

BLEU по формуле

BLEU одного сегмента в том виде, в каком его считает Л.р. № 3

$$p_n = \frac{m_n + 1}{t_n + 1}, \qquad n = 1, 2, 3, 4$$

$$\text{BP} = \begin{cases} 1, & c > r, \\ e^{\,1 - r/c}, & c \le r \end{cases}$$

$$\text{BLEU} = \text{BP} \cdot \sqrt[k]{p_1 \, p_2 \cdots p_k}$$

t_n — сколько n-грамм слов в гипотезе; m_n — сколько из них есть в эталоне (каждая — не больше раз, чем там встречается).

c и r — длина гипотезы и эталона в словах. BP (brevity penalty) — штраф только за слишком короткий перевод.

k — сколько долей удалось посчитать: в гипотезе из двух слов нет ни 3-грамм, ни 4-грамм.

+1 в числителе и знаменателе — сглаживание. Без него одна нулевая доля обнуляет весь BLEU: произведение с нулём — ноль. Слова сравниваются в нижнем регистре, пунктуация отбрасывается.

BLEU вручную: откуда 0,779 и 0,408

«Удалось подключиться к серверу»
эталон: «Не удалось подключиться к серверу»

n	совпало, m_n/t_n	p_n
1	4 из 4	$5/5 = 1$
2	3 из 3	$4/4 = 1$
3	2 из 2	$3/3 = 1$
4	1 из 1	$2/2 = 1$

Среднее долей — 1. Но $c = 4$, $r = 5$:
$\mathrm{BP} = e^{1-5/4} = e^{-0{,}25} = \mathbf{0{,}779}$.

«Сохранение изменений»
эталон: «Сохранить изменения»

n	совпало, m_n/t_n	p_n
1	0 из 2	$1/3$
2	0 из 1	$1/2$
3, 4	в гипотезе таких нет	

$c = r = 2$, $\mathrm{BP} = 1$:
$\mathrm{BLEU} = \sqrt{\frac{1}{3} \cdot \frac{1}{2}} = \sqrt{\frac{1}{6}} = \mathbf{0{,}408}$.

Слева совпали все n-граммы, и весь штраф — за длину: потерю «не» BLEU видит только как недостающее слово. Справа не совпало ни одного слова, а BLEU всё равно 0,408 — это сглаживание, а не сходство.

chrF и почему для русского он подходит лучше

chrF сравнивает символьные n-граммы и считает F-меру — точность вместе с полнотой. У «сохранить» и «сохранение» общая основа, и словоизменение перестаёт обнулять совпадение.

	BLEU	chrF
другие формы	0,408	0,602
потеряно отрицание	0,779	**0,919**

Лучше — не значит безопасно. Потерянное отрицание chrF замечает ещё хуже BLEU: пропала одна короткая частица.

Ещё две метрики, о которых стоит знать: TER — сколько правок нужно постредактору; COMET — нейросеть, обученная на человеческих оценках.

→ **метрики машинного перевода** — BLEU и chrF одного сегмента, с учётом морфологии и без.

chrF по формуле: точность, полнота, F-мера

Символьные n-граммы, $n = 1, \ldots, 6$; пробел — тоже символ; m_n — сколько совпало:

$$P_n = \frac{m_n}{\text{в гипотезе}}, \qquad R_n = \frac{m_n}{\text{в эталоне}}$$

$$F_n = \frac{(1+\beta^2)\, P_n R_n}{\beta^2 P_n + R_n}, \quad \beta = 2$$

$$\text{chrF} = \tfrac{1}{6}\,(F_1 + \ldots + F_6)$$

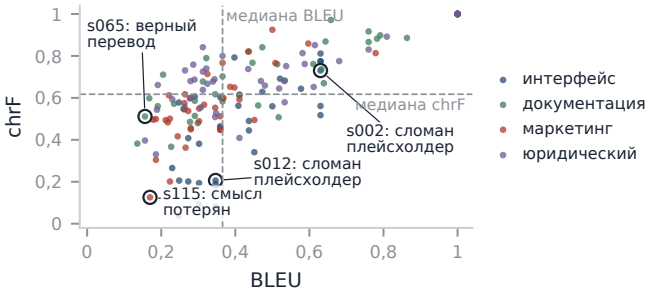
Точность P — какая доля сказанного верна (не путать с точностью классификатора); полнота R — какая доля нужного сказана. $\beta = 2$: полнота вдвое важнее.

«Сохранение изменений» против эталона «Сохранить изменения»

n	m_n	P_n	R_n	F_n
1	16	16/20	16/19	0,833
2	14	14/19	14/18	0,769
3	11	11/18	11/17	0,640
4	9	9/17	9/16	0,556
5	7	7/16	7/15	0,461
6	5	5/15	5/14	0,352
среднее — chrF				**0,602**

С потерянным «не» точность у chrF равна 1 на всех n, а полнота — около 0,9: недостаёт трёх символов «не␣» из 33. Итог 0,919 — метрика почти не заметила отрицания.

BLEU и chrF на всём корпусе



Точка — сегмент машинного перевода Л.р. № 3. В целом метрики согласны: ранговая корреляция **0,73**.

Но ловят не всё: сломанный s012 и искажённый s115 — внизу, а сломанный s002 — выше обеих медиан. Верный s065 — ниже обеих.

Медианы — 0,366 и 0,617. Порог по любой отсекает половину корпуса, не отличая сломанное от перефразированного.

Слепые зоны: что метрика не поймает никогда

что случилось	пример из Л.р. № 3	BLEU
сломан плейсхолдер	s002: `Удалить файл "(имя}"?` — строка не соберётся	0,630
потеряно отрицание	«Удалось подключиться» вместо «Не удалось»	0,779
верный перевод	s065: «Приведенный ниже пример показывает…» вместо «В следующем примере показана…»	0,156

Медиана BLEU по корпусу — **0,366**. Приёмка по ней пропустила бы сломанную строку и отклонила безупречный перевод. Причина одна: метрики сравнивают форму, и смысл, перевёрнутый крошечной правкой формы, они не видят *в принципе* — так они устроены. Это не ошибка в программе.

Чем закрывают слепые зоны метрик

Раз метрика слепа по самому своему устройству, её не чинят — её дополняют.

средство	на проверку	доля
BLEU ниже медианы	78	48,8 %
chrF ниже медианы	80	50,0 %
семантическая близость $< 0{,}7$	15	9,4 %
формальные проверки	**8**	**5,0 %**

160 сегментов, Л.р. № 3, раздел 8.

Формальные проверки (плейсхолдеры, числа, отрицание, длина) ловят механические ошибки — все до одной. Сломанный s002 нашли *только* они: BLEU у него выше медианы, семантическая близость — 0,801.

Средство ценно не полнотой, а отношением найденного к просмотренному. Порог по метрике отправляет на проверку половину корпуса.

Когда метрике можно верить

Можно

Сравнить две системы перевода на тысячах сегментов. Отследить, не ухудшилась ли модель после обновления. Отобрать сегменты, заведомо требующие внимания.

Нельзя

Принять или отклонить отдельный сегмент. Оценить качество на коротких строках интерфейса. Заменить метрикой приёмку человеком.

Вопрос заказчика из задания 9 Л.р. № 3: «можно ли принимать перевод, если BLEU выше порога?» Ответ — нет, и после этого блока вы можете объяснить почему, а не сослаться на интуицию. Это и есть профессиональная позиция, которую даёт курс.

БЛОК 5

Данные и разметка

Во что упирается качество, сколько стоит разметка и насколько ей можно верить. Это та часть работы, которую делает лингвист.

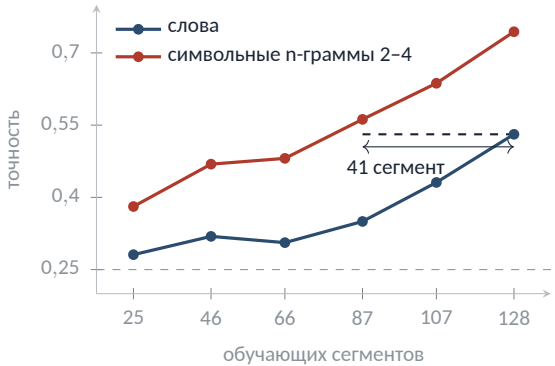
Во что упирается качество

Пять очевидных ходов и что они дают (Л.р. № 1, разделы 8, 9 и 13):

ход	результат	почему
подобрать гиперпараметры, 48 комбинаций	+0,000	нашлась ровно исходная
склеить слова и n-граммы	−0,019	больше признаков — больше шума
сменить модель	0,550–0,744	разброс 0,19
сменить предобработку	0,381–0,631	разброс 0,25
разметить больше, 20 Newsgroups	**0,546 → 0,830**	в 11 раз больше примеров

Порядок при нехватке качества: данные, потом признаки, потом модель. Подбор гиперпараметров даёт ноль, модель и признаки — величины одного порядка, данные — десятки процентов.

Кривая обучения читается и по горизонтали



Логистическая регрессия, пятикратная кросс-валидация.

Символьные n-граммы на **87** сегментах уже обгоняют слова, обученные на всех **128**. Удачная идея о признаках экономит 41 сегмент разметки.

Обе кривые на правом краю ещё растут: новые размеченные данные ещё помогут.

→ **данные и разметка** — кривые считаются в браузере, модель переключается.

MQM: отраслевая схема оценки перевода

категория	что означает	пример
точность	смысл искажён, добавлен или потерян	пропало предложение
беглость	нарушены нормы языка перевода	сбой согласования
терминология	не тот термин, что принят в проекте	«любимые» вместо «избранное»
стиль	нарушен регистр или тон	«ты» вместо «вы»
разметка	сломаны плейсхолдеры, теги	`{name}` переведён

Серьёзность и вес: критическая — **25**, серьёзная — **5**, незначительная — **1**, нет — 0. Критическая — продукт сломан или смысл противоположный.

Веса неравномерны намеренно: одна критическая ошибка «стоит» двадцати пяти незначительных. Они отражают стоимость последствий, а не частоту. Л.р. № 3, раздел 6.

Метрика против разметки

Десять сегментов, размеченных в разделе 6 Л.р. № 3

сегмент	ошибка	BLEU
s002	критическая, разметка	0,630
s006	незначительная, термин	0,630
s012	критическая, разметка	0,347
s115	критическая, точность	0,170
s065	**нет**	0,156

Пять из десяти; остальные — на интерактивной странице.

Критическая и незначительная ошибки получили *одинаковый* BLEU. Безупречный перевод — самый низкий из десяти.

Ранговая корреляция BLEU и штрафа MQM — $+0{,}18$: связи нет ни в какую сторону. Порога BLEU, при котором приёмка на этих десяти сегментах не ошибается, не существует.

→ **данные и разметка** — порог приёмки двигается ползунком, ошибки считаются сразу.

Согласие разметчиков

Разметка — это измерение, и у измерения есть надёжность. Если два человека размечают один материал по-разному, ненадёжна и разметка, и всё, что из неё выведено.

Каппа Коэна — согласие за вычетом случайного:

$$\kappa = \frac{p_{\text{совп}} - p_{\text{случ}}}{1 - p_{\text{случ}}}$$

Пример Л.р. № 3: совпали 8 ответов из 10, случайно совпало бы 22 %: $\kappa = \frac{0{,}80 - 0{,}22}{1 - 0{,}22} \approx 0{,}744$ — «существенное» согласие.

Зачем вычитать случайные совпадения. Если 80 % сегментов без ошибок, два разметчика, которые вообще не согласны, совпадут в **65 %** случаев — просто потому, что оба чаще ставят «нет». Каппа у них — ноль.

Расхождения примера — s003: точность или терминология? s006: терминология или стиль? Оба ответа можно обосновать, пока руководство для разметчиков не решило иначе. Написать его — профессиональная задача (Л.р. № 3, задание 8).

Права, персональные данные и загрязнение корпусов

Права на корпус

Тексты из интернета не становятся свободными оттого, что доступны. У OPUS, Universal Dependencies и подобных ресурсов лицензии указаны явно — их надо читать.

Персональные данные

Переписка, тикеты поддержки и отзывы содержат имена, адреса, номера. Обезличивание — обязательный этап, а не пожелание.

Следы машинного перевода

Заметная часть параллельных корпусов из сети сама получена машинным переводом. Модель воспроизводит чужие ошибки, а оценка на таких данных завышается.

Умение сказать заказчику, что с корпусом что-то не так, — часть профессии, и её не заменит никакая метрика.

Что связывает все пять блоков

Машинное обучение не работает с языком — оно работает с числами, полученными из языка. Качество этого перехода определяет всё остальное.

Три вещи, которые делает лингвист и не может сделать инженер

Решает, что считать единицей и как её нормализовать.

блоки 1–2

Решает, что считать ошибкой, и пишет об этом руководство.

блоки 4–5

Читает результат и объясняет, почему модель ошиблась именно так.

все лабораторные

Всё это можно измерить, и в лабораторных вы это сделаете.

Для желающих углубиться: D. Jurafsky, J. H. Martin. *Speech and Language Processing*, 3-е изд., свободно доступно онлайн.